

PQC IKEv2: Explicit vs. Implicit Authentication Analysis

Quantitative Analysis of draft-wang-ipsecme-kem-auth-ikev2-03

Uri Blumenthal

Prepared for: CFRG and IPSECME Review

Date: 2026-06-16 (v27a)

Status: Technical Comparison — IETF IPSECME and CFRG Distribution

Executive Summary

This document quantifies the performance, bandwidth, memory, and exchange-count trade-offs between two approaches to post-quantum mutual authentication in IKEv2:

- **Existing-Explicit (Dual-Signature):** Classical ECDHE in `IKE_SA_INIT` + ephemeral PQ KEM in `IKE_INTERMEDIATE` (per RFC 9370) + two digital signatures in `IKE_AUTH`.
- **Proposed-Implicit (Dual-KEM):** Same hybrid ECDHE + ephemeral PQ KEM structure, but authentication signatures are replaced by dual KEM encapsulations (draft-wang-ipsecme-kem-auth-ikev2-03, following MQV/HMQV/KEMTLS family of protocols).

Note: Both approaches preserve the classical ECDHE exchange in `IKE_SA_INIT`, yielding a **hybrid classical + PQ construction**. The PQ KEM payload is carried in `IKE_INTERMEDIATE` per RFC 9370, not in `IKE_SA_INIT`, because the ML-KEM-768 public key (1 184 B) plus response ciphertext (1 088 B) exceed safe UDP MTU budgets in the hybrid scenario and cannot be fragmented at the `IKE_SA_INIT` stage.

Two variants of the implicit approach are analyzed:

- **Proposed-A** ML-KEM-768 for both ephemeral and authentication KEM;
- **Proposed-B** ML-KEM-768 ephemeral + Classic McEliece (with preloaded static keys or certificates) for authentication.

In the broader KEM-authentication literature, the specific idea of using **Classic McEliece as the long-term/authentication KEM when keys are pre-distributed** is already present in **KEMTLS-PDK** [22], which explicitly notes that Classic McEliece becomes suitable for authentication in the pre-distributed-key setting because its very large public key stays off the wire while its ciphertext remains small.

Bottom line: Proposed-A cuts per-peer CPU time by ~63% and reduces wire bytes by 44–64% (64% with preloaded certificates). Proposed-B saves 78% of wire bytes with modest CPU overhead and provides Post-Quantum algorithm diversity. Neither implicit variant — neither Proposed-A nor Proposed-B — reduces the exchange count relative to a compatible PQ-signature deployment; both require one *additional* exchange (see Section 8).

The approach inherits from the MQV idea, and is closely related **KEMTLS** [21, 26], **AuthKEM** [24], and **Muckle#** [34]. KEMTLS/AuthKEM apply the same dual-KEM implicit-authentication principle to TLS 1.3; Muckle# extends KEM-based authentication into a hybrid AKE framework that combines conventional, post-quantum, and QKD-derived key material.

Contents

1	Introduction	3
2	Algorithm Primitive Benchmarks	3
2.1	Reference Platform	3
2.2	Per-Operation Latency	4
3	Key and Message Sizes	6
4	Protocol Comparison	7
4.1	Notes on Diagram Scope and ML-KEM in IKE_SA_INIT	7
4.2	Existing: Explicit Authentication via Dual Signatures	7
4.3	Proposed: Implicit Authentication (draft-wang-ipsecme-kem-auth-ikev2-03) . .	8
4.4	Per-Peer Operation Count Comparison	12
4.5	Backward Compatibility Considerations	12
5	Performance Comparison	12
6	Wire Bytes Comparison	14
7	Implementation Memory Requirements	18
8	Protocol Exchange Count Comparison	19
8.1	Analysis	19
8.2	Exchange Count (1 exchange = 1 RT)	19
9	Summary Comparison Tables	21
10	Security Analysis	22
10.1	Explicit Authentication Security	22
10.2	Implicit Authentication — MQV/HMQV Lineage	23
10.3	Non-Repudiation Trade-off	24
10.4	McEliece-Specific Security Diversity (Proposed-B)	24
10.5	Identity Privacy and the Cost of Combining Exchanges	25
10.6	Failure Modes and Error Handling	26
11	Conclusions	27
	References	29
A	Benchmark Data Sources	32
A.1	ML-DSA and FN-DSA: Local liboqs Measurements	32
A.2	Raw Benchmark Output	32
A.3	Interpretation and Usage	33
A.4	Other Data Sources	33

1. Introduction

Post-quantum migration for IKEv2 requires replacing the classical signature-based authentication model with mechanisms that remain secure against quantum adversaries while preserving interoperability with existing deployments. The “Proposed” solution analyzed in this document is based on **draft-wang-ipsecme-kem-auth-ikev2-03** [5], which introduces a KEM-based implicit authentication method for IKEv2.

This approach is part of a broader family of work on implicit authentication using KEMs. In TLS, the KEMTLS protocol (Stebila-Schwabe-Wiggers, CCS 2020 [21]) and its IETF standardization (AuthKEM [24]) demonstrate that static KEM keys embedded in certificates can provide mutual authentication without signatures. The PQuAKE protocol [20] applies similar ideas to authenticated key exchange in the CFRG Research Group. Muckle# [34] takes a closely related HAKE route: it replaces post-quantum signatures with KEM-based authentication in a hybrid AKE that combines conventional, post-quantum, and QKD key material. The IKEv2 KEM-Auth draft builds on this broader lineage. All of these efforts trace back to the MQV/HMQV family of implicit-authentication protocols [18, 19].

Importantly, per RFC 7296 [1] and RFC 9370 [2], the Proposed protocol **preserves the classical ECDHE exchange in IKE_SA_INIT** and carries the PQ KEM data in **IKE_INTERMEDIATE**. The result is a **hybrid classical + post-quantum key exchange** whose session keys derive from both classical ECDHE and post-quantum KEM shared secrets.

2. Algorithm Primitive Benchmarks

2.1 Reference Platform

All cycle counts and latency figures are normalized to **Intel x86-64 with AVX2 at 3.0 GHz**. Where measurements were taken on different hardware, they are scaled using cycle counts directly (cycles are clock-speed-independent on out-of-order superscalar CPUs).

Primary sources:

- wolfSSL Manual Appendix G — wolfCrypt benchmarks, Intel x86-64 AVX2, averaged over 1,000 runs [13].
- pq-crystals/dilithium — SUPERCOP cycle counts, Intel KabyLake (~3.6 GHz AVX2), scaled to 3.0 GHz [14].
- **libmciece speed table** (official Classic McEliece implementation, 2025-05-06): interquartile means of single-core cycle counts on Golden Cove (2021) [12]. All McEliece figures use Golden Cove data. *For McEliece specifically*, the PQClean/liboqs implementation is substantially slower and is not used here.
- Frontiers article — ML-KEM-1024 + ML-DSA-65 on Intel Core i7-12700K (3.6 GHz), 1,000-run average [17].
- **Local liboqs measurements** (Appendix section A): ML-DSA and FN-DSA benchmark data from Open Quantum Safe liboqs version 0.8.1-dev on x86_64-Darwin-22.5.0 (clang 16, AVX2 enabled) [10]. Used as the measured implementation data point for ML-DSA and FN-DSA rows throughout this document.

2.2 Per-Operation Latency

ML-DSA benchmark note. This revision shows **two ML-DSA benchmark views**: an **optimized AVX2 estimate** derived from pq-crystals/SUPERCOP-style cycle counts and a **measured liboqs/macOS view** from the local benchmark measurements documented in Appendix [section A](#) (Dilithium2/3/5, i.e., ML-DSA-44/65/87). The protocol-comparison tables in [section 5](#) retain the optimized AVX2 ML-DSA-65 baseline for continuity with earlier drafts, and also add an explicit measured-ML-DSA comparison table so readers can see the spread between a tuned lower-bound estimate and a concrete shipping-library build.

Conversion: latency (μs) = cycles $\div$ 3 000. All values are in μs . McEliece KeyGen values are large but performed offline and carry **zero** per-session cost.

Table 1: Per-operation latency (normalized to 3.0 GHz Intel x86-64 AVX2).

Algorithm	Op.	Cycles	μs	Source / Notes
ML-KEM-512	KeyGen	$\sim 14\,960$	5	wolfSSL AVX2
	Encap	$\sim 16\,650$	6	wolfSSL AVX2
	Decap	$\sim 19\,090$	6	wolfSSL AVX2
ML-KEM-768	KeyGen	$\sim 18\,000$	6	wolfSSL AVX2
	Encap	$\sim 21\,000$	7	wolfSSL AVX2
	Decap	$\sim 30\,000$	10	wolfSSL AVX2
ML-KEM-1024	KeyGen	$\sim 27\,000$	9	wolfSSL AVX2
	Encap	$\sim 30\,000$	10	wolfSSL AVX2
	Decap	$\sim 42\,000$	14	wolfSSL AVX2
ML-DSA-44 [opt. AVX2]	KeyGen	$\sim 44\,000$	15	pq-crystals AVX2 scaled
	Sign	$\sim 87\,000$	29	pq-crystals AVX2 scaled
	Verify	$\sim 52\,000$	17	pq-crystals AVX2 scaled
ML-DSA-44 [measured liboqs]	KeyGen	65 563	21.9	Appendix A (Dilithium2)
	Sign	167 361	55.8	Appendix A (Dilithium2)
	Verify	61 392	20.5	Appendix A (Dilithium2)
ML-DSA-65 [opt. AVX2]	KeyGen	$\sim 60\,000$	20	pq-crystals AVX2 scaled
	Sign	$\sim 135\,000$	45	pq-crystals AVX2 scaled
	Verify	$\sim 73\,000$	24	pq-crystals AVX2 scaled
ML-DSA-65 [measured liboqs]	KeyGen	112 440	37.5	Appendix A (Dilithium3)
	Sign	272 595	90.9	Appendix A (Dilithium3)
	Verify	101 799	33.9	Appendix A (Dilithium3)
ML-DSA-87 [opt. AVX2]	KeyGen	$\sim 90\,000$	30	pq-crystals AVX2 scaled

Algorithm	Op.	Cycles	μ s	Source / Notes
	Sign	~180 000	60	pq-crystals AVX2 scaled
	Verify	~105 000	35	pq-crystals AVX2 scaled
ML-DSA-87 [measured liboqs]	KeyGen	190 290	63.4	Appendix A (Dilithium5)
	Sign	362 451	120.8	Appendix A (Dilithium5)
	Verify	164 055	54.7	Appendix A (Dilithium5)
FN-DSA-512	KeyGen	16 627 157	5 542	Appendix A (Falcon-512); normalized to 3.0 GHz
	Sign	635 726	212	Appendix A (Falcon-512); normalized to 3.0 GHz
	Verify	118 375	39	Appendix A (Falcon-512); normalized to 3.0 GHz
FN-DSA-1024	KeyGen	49 088 028	16 363	Appendix A (Falcon-1024); normalized to 3.0 GHz
	Sign	1 308 057	436	Appendix A (Falcon-1024); normalized to 3.0 GHz
	Verify	235 577	79	Appendix A (Falcon-1024); normalized to 3.0 GHz
McEliece-348864	KeyGen (f)	~27 700 000	9 230	libmceliece, Golden Cove; offline
	KeyGen	~48 900 000	16 300	libmceliece, Golden Cove; offline
	Encap	~25 100	8	libmceliece, Golden Cove
	Decap	~104 300	35	libmceliece, Golden Cove
McEliece-460896	KeyGen (f)	~88 000 000	29 300	libmceliece, Golden Cove; offline
	KeyGen	~120 900 000	40 300	libmceliece, Golden Cove; offline
	Encap	~55 200	18	libmceliece, Golden Cove
	Decap	~226 500	76	libmceliece, Golden Cove
McEliece-6688128	KeyGen (f)	~195 700 000	65 200	libmceliece, Golden Cove; offline
	KeyGen	~364 800 000	121 600	libmceliece, Golden Cove; offline
	Encap	~95 100	32	libmceliece, Golden Cove
	Decap	~257 100	86	libmceliece, Golden Cove

McEliece KeyGen: Performed offline, once per long-term identity key. Per-session cost is zero. The (f) variant uses a faster seeded algorithm.

FN-DSA KeyGen: Similarly offline; NTRU lattice basis generation costs 3.3–6.7 ms at 3.0 GHz [32, 9].

ML-DSA Sign variance: Rejection sampling causes high latency variance (coefficient of variation, CV, 66–73%; 99th-percentile up to 4× mean; per Halak et al. 2025 [16]).

Performance Variance. The cycle counts in Table 1 represent **average-case** performance. Real-world latency exhibits variance due to:

- **ML-DSA rejection sampling:** Sign operations use rejection sampling to ensure security. Each iteration has probability ~50% of rejection. This causes high variance: the coefficient

of variation (CV) is 66–73%, and 99th percentile latency can reach $4\times$ the mean [16].

- **Falcon Gaussian sampling:** KeyGen exhibits high variance (CV $\sim 80\%$) due to probabilistic lattice basis construction. Sign variance is lower (CV $\sim 30\%$) but still significant [9, 32].
- **ML-KEM:** Deterministic operations with negligible variance ($<5\%$, due to cache effects only) [7].
- **McEliece:** Deterministic; variance $<3\%$ [11].

For high-throughput VPN gateways performing capacity planning, 99th-percentile latency is more relevant than mean. A gateway must be provisioned so that it keeps up under worst-case, not average, load: when many handshakes arrive concurrently, a high-variance operation produces a long tail of slow handshakes that queue behind one another, so the tail (p99), rather than the mean, determines the per-core handshake rate that can be sustained without backlog, the latency budget for connection-setup SLAs, and the headroom needed to absorb bursts. Deterministic operations with a short tail can therefore be provisioned much closer to their mean cost:

- ML-DSA-65 Sign: mean $45\ \mu s$, p99 $\sim 180\ \mu s$ ($4\times$)
- FN-DSA-512 Sign: mean $212\ \mu s$, p99 $\sim 600\ \mu s$ ($2.8\times$)
- ML-KEM-768 Encap/Decap: mean $7/10\ \mu s$, p99 $\sim 8/11\ \mu s$ ($1.1\times$)

The Proposed protocol’s use of deterministic KEM operations eliminates tail-latency concerns present in signature-based approaches, improving predictability for real-time systems and capacity planning.

3. Key and Message Sizes

Table 2: Cryptographic object sizes (bytes). Sources: FIPS 203 [7], 204 [8], 206 [9]; McEliece Round 4 [11].

Algorithm	Public Key (B)	Secret Key (B)	CT / Sig (B)
ML-KEM-512	800	1 632	768 (ct)
ML-KEM-768	1 184	2 400	1 088 (ct)
ML-KEM-1024	1 568	3 168	1 568 (ct)
ML-DSA-44	1 312	2 528	2 420 (sig)
ML-DSA-65	1 952	4 032	3 309 (sig)
ML-DSA-87	2 592	4 896	4 595 (sig)
FN-DSA-512	897	1 281	666 (sig, avg)
FN-DSA-1024	1 793	2 305	1 280 (sig, avg)
McEliece-348864	261 120	6 492	96 (ct)
McEliece-460896	524 160	13 568	156 (ct)
McEliece-6688128	1 044 992	13 892	208 (ct)

4. Protocol Comparison

4.1 Notes on Diagram Scope and ML-KEM in IKE_SA_INIT

The sequence diagrams below depict logical cryptographic operations. They do not reflect the complete IKEv2 payload structure (notify types, SA payload formats, etc.), as the main purpose is comparison between the elements that differ between this draft and the current design.

Placement of ML-KEM in the existing approach: Draft-ietf-ipsecme-ikev2-mlkem-03 [4] specifies that ML-KEM-512 and ML-KEM-768 *may* be used directly in `IKE_SA_INIT` as the sole (non-hybrid) key exchange. This direct placement is practical only when the path MTU comfortably accommodates the ML-KEM public key and ciphertext together with the other `IKE_SA_INIT` payloads (e.g., on links with a 1500-byte MTU). Otherwise the messages exceed the MTU and incur IP-level fragmentation, which IKEv2 message fragmentation (RFC 7383 [3]) cannot remedy in the initial exchange — fragmentation support is itself negotiated *within* `IKE_SA_INIT` and so is not yet available for that exchange. In that case the key-exchange data must instead be carried in `IKE_INTERMEDIATE` per RFC 9370. However, in the standard hybrid deployment — where classical ECDHE is retained for backward compatibility and ML-KEM is the *additional* PQ KEM — RFC 9370 [2] requires that all additional key exchange data be carried in `IKE_INTERMEDIATE`, not in `IKE_SA_INIT`. Figure 1 reflects this hybrid-mode structure.

Hybrid security: The classical ECDHE exchange in `IKE_SA_INIT` remains unchanged in both the existing and proposed approaches.

Protection of the `IKE_INTERMEDIATE` exchanges: The sequence diagrams omit the IKEv2 key schedule for clarity, but it is worth stating explicitly that every `IKE_INTERMEDIATE` message is already cryptographically protected. Before the first `IKE_INTERMEDIATE` is sent, IKEv2 has derived `SKEYSEED` and the working keys (`SK_e`/`SK_a`/`SK_d`) from the `IKE_SA_INIT` exchange — that is, from the classical ECDHE shared secret SS_{cl} together with the exchanged nonces. Per RFC 9370 [2], the key schedule (specifically `SK_d`) is then *updated* after each additional key exchange so as to fold in the new shared secret before the next exchange; the explicit `SK_d` recurrence is given in the PPK discussion of Section 4.3.

4.2 Existing: Explicit Authentication via Dual Signatures

Per draft-ietf-ipsecme-ikev2-mlkem-03 and RFC 9370 in hybrid mode: classical ECDHE (`KEi`/`KEr`) in `IKE_SA_INIT`; ML-KEM-768 ephemeral PK/CT in `IKE_INTERMEDIATE`; certificate and signature in `IKE_AUTH`.

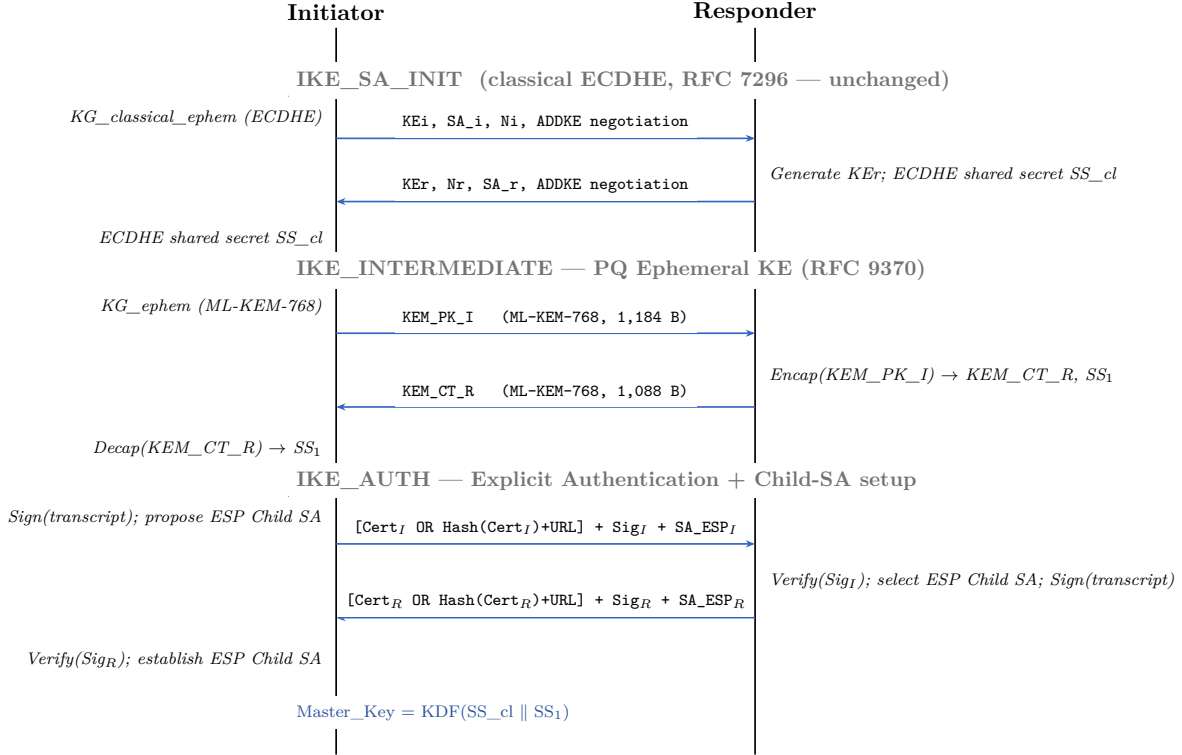


Figure 1: Existing: hybrid ECDHE + ML-KEM-768 key exchange with explicit dual-signature authentication. Per RFC 9370 hybrid mode, the classical ECDHE stays in **IKE_SA_INIT**; the ML-KEM ephemeral PK/CT moves to **IKE_INTERMEDIATE**. **IKE_AUTH** also carries **ESP Child-SA negotiation**, so the comparison with the proposed Key-Confirmation (KC) piggybacking design is like-for-like. **Certificate options:** Each peer sends either (a) full X.509 certificate containing ML-DSA-65 public key ($\sim 1,952$ B pk + ~ 200 B wrapper), or (b) 20-byte SHA-1 hash of preloaded certificate ($Hash(Cert_I)/Hash(Cert_R)$, per RFC 7296 CERTREQ), for a fairer comparison with the proposed implicit authentication scheme. (Here and in Figure 2, $KG_classical$ denotes the complete classical ECDHE contribution — ephemeral key generation *and* the shared-secret agreement, i.e. both scalar multiplications — which is identical across all approaches and is excluded from the latency tables of Section 5.) **Initiator:** $KG_classical + KG_ephem + Decap(SS_1) + Sign + Verify$. **Responder:** $KG_classical + Encap(SS_1) + Sign + Verify$.

Note: We assume that each peer sends one certificate, rather than a certificate chain. If a set of intermediate CAs has to be included, it would greatly increase the number of bytes over the wire for both implicit and explicit authentication approaches.

4.3 Proposed: Implicit Authentication (draft-wang-ipsecme-kem-auth-ikev2-03)

Note: Here and in subsequent diagrams, derivation of the $Master_Key$ has been simplified to avoid distractions from the purpose of this analysis.

The Proposed protocol (depicted on Figure 2) is the IKEv2 adaptation of the implicit-authentication concept pioneered by MQV and HMQV [18, 19], formalized for TLS in KEMTLS [21], brought to the IETF in AuthKEM [24], and independently developed in a hybrid-AKE/QKD setting

in Muckle# [34]. Static KEM keys embedded in certificates (or pre-distributed) replace signatures for entity authentication; shared secrets from auth-KEM encapsulations are folded into the key schedule to bind the session to authenticated identities [20, 34]. The Key Confirmation (KC) exchange is directly analogous to the KEMTLS/AuthKEM Finished messages and to Muckle#’s final MAC exchange [21, 24, 20, 34].

Certificate Transmission Timing. The responder’s KEM certificate (or its 20-byte SHA-1 fingerprint) **MUST** be transmitted in the **first** `IKE_INTERMEDIATE` exchange (PQ ephemeral KE), not the second (auth-KEM) exchange. This is required because the initiator must know the responder’s static KEM public key before it can compute $CT_{auth,I}$ in the auth-KEM exchange.

Certificate options:

- **Full certificate:** $Cert_R$ transmitted in first `IKE_INTERMEDIATE` (ML-KEM-768 pk 1,184 B + X.509 wrapper ~200 B for Proposed-A; McEliece-460896 pk 524,160 B + wrapper for Proposed-B).
- **Preloaded certificate:** $Hash(Cert_R) = 20\text{-byte SHA-1 fingerprint}$ (per RFC 7296 CERTREQ payload conventions) transmitted in first `IKE_INTERMEDIATE`.

The initiator’s certificate (or fingerprint) is transmitted in the **second** `IKE_INTERMEDIATE` (auth-KEM exchange) along with $CT_{auth,I}$.

Operational Flow. The step-by-step split between encapsulation, decapsulation, key derivation, and explicit key confirmation follows the same structural pattern as PQuAKE [20], adapted here to IKEv2 exchange semantics.

Initiator operations:

- ECDHE KeyGen (classical component, `IKE_SA_INIT`)
- ML-KEM-768 ephemeral KeyGen (`IKE_INTERMEDIATE` exchange 1)
- Auth-KEM Encap to responder’s static PK (`IKE_INTERMEDIATE` exchange 2)
- Ephemeral KEM Decap to derive SS_1 (from exchange 1)
- Auth-KEM Decap to derive $SS_{auth,R}$ (from exchange 2)
- HMAC(KC_I) computation and verification of KC_R (`IKE_AUTH`)

Responder operations:

- ECDHE KeyGen (classical component, `IKE_SA_INIT`)
- Ephemeral KEM Encap to initiator’s ephemeral PK (`IKE_INTERMEDIATE` exchange 1)
- Auth-KEM Decap using own static SK (from exchange 2)
- Auth-KEM Encap to initiator’s static PK (`IKE_INTERMEDIATE` exchange 2)
- Verification of KC_I and HMAC(KC_R) (`IKE_AUTH`)

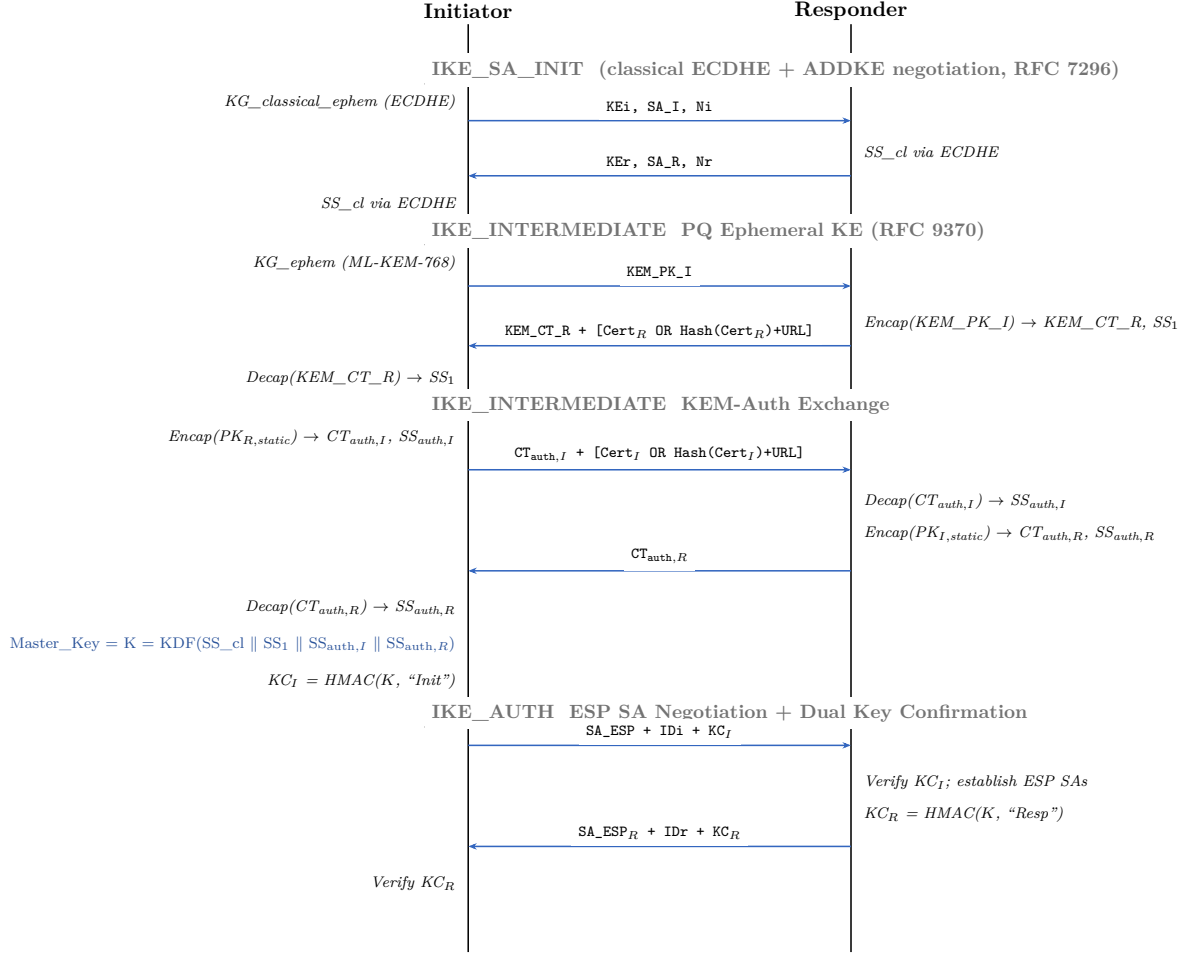


Figure 2: Proposed: implicit authentication (dual-KEM). Classical ECDHE in **IKE_SA_INIT**; PQ ephemeral KEM in **IKE_INTERMEDIATE**; KEM-Auth encapsulations in a second **IKE_INTERMEDIATE**; both KC values and ESP SA setup in **IKE_AUTH**. **Certificate options:** Each peer sends either (a) full KEM certificate ($Cert_I/Cert_R$: ML-KEM-768 pk 1,184B + X.509 wrapper ~ 200 B), or (b) 20-byte SHA-1 hash of preloaded certificate ($Hash(Cert_I)/Hash(Cert_R)$, per RFC 7296 CERTREQ). Responder additionally transmits its certificate (or hash) together with KEM_CT_R in the PQ ephemeral **IKE_INTERMEDIATE** so the initiator knows $PK_{R,static}$ for $CT_{auth,I}$. Both KC_I and KC_R are computed after the Master_Key is established and exchanged in **IKE_AUTH**, matching the protocol as specified in draft-wang-ipsecme-kem-auth-ikev2-03. **Note on identity payloads:** Per RFC 7296, IKEv2 Identity Payloads (ID_i/ID_r) are transmitted only in **IKE_AUTH**; they are *not* sent as separate payloads in **IKE_INTERMEDIATE**. However, KEM certificates carried in **IKE_INTERMEDIATE** (or referenced via Hash+URL) embed identity information, so peer identities are *indirectly* communicated at that stage when certificates are exchanged on the wire. **Initiator:** $KG_classical + KG_ephem + Auth-Encap$ (to peer's static PK) + $Decap(SS_1)$ + $Auth-Decap$ (from peer) + $HMAC(KC_I)$ + $Verify(KC_R)$. **Responder:** $Encap(SS_1)$ + $Auth-Decap$ (from initiator) + $Auth-Encap$ (to initiator's static PK) + $Verify(KC_I)$ + $HMAC(KC_R)$.

Post-Quantum Preshared Key (PPK) Integration. Incorporating pre-shared symmetric keys into key-exchange mechanisms as an additional layer of defence against a passive

adversary storing traffic for later CRQC-assisted decryption has been explored in the TLS context (see [25]); the same principle applies directly to IKEv2 via the PPK mechanism. Per RFC 9867 [29], IKEv2 supports mixing a preshared key (PPK) into the key schedule at the `IKE_INTERMEDIATE` stage for additional quantum resistance. In the Proposed protocol, PPK integration occurs as follows:

- **Negotiation:** PPK use is signaled in `IKE_SA_INIT` via the `USE_PPK` notify.
- **Timing:** PPK is applied in the **first** `IKE_INTERMEDIATE` (ephemeral PQ KE exchange), not in `IKE_AUTH`. This is critical: it ensures that the auth-KEM exchange (second `IKE_INTERMEDIATE`) and the long-term KEM certificates are protected by the PPK-derived keying material.
- **Key schedule:** If PPK is negotiated, the ephemeral key derivation becomes:

$$SK_{d,new} = KDF(SK_{d,old}; SS_{cl} || SS_1 || PPK)$$

where the KDF is invoked under key $SK_{d,old}$ (the existing IKEv2 traffic-derivation key) and the concatenation $SS_{cl} || SS_1 || PPK$ is the input keying material. The updated $SK_{d,new}$ is then used to derive encryption keys for subsequent `IKE_INTERMEDIATE` messages, including the auth-KEM exchange.

- **Security impact:** PPK provides an additional quantum-resistant secret, independent of both ECDHE and ML-KEM. This creates a **triple-layer** quantum defense: ephemeral ML-KEM + PPK + auth-KEM. Critically, applying PPK at the `IKE_INTERMEDIATE` stage (per RFC 9867) protects the auth-KEM ciphertexts and long-term KEM certificates against a passive adversary with a CRQC, even if the ephemeral ML-KEM is later broken.
- **Exchange count:** PPK does not add exchanges; it is incorporated into the existing key schedule at the first `IKE_INTERMEDIATE` stage and can be piggybacked with the ephemeral PQ KE payloads.

Deployments requiring maximum quantum resistance **SHOULD** use PPK (per RFC 9867) in addition to the Proposed protocol, not as a substitute. RFC 8784 [28] (PPK in `IKE_AUTH`) is unsuitable for protecting auth-KEM exchanges, as the PPK is applied too late in the handshake.

Key Confirmation Timing. Both KC_I and KC_R are computed *after* the `Master_Key` is derived from all four shared secrets (SS_{cl} , SS_1 , $SS_{auth,I}$, $SS_{auth,R}$). This ensures that key confirmation validates the complete key schedule, including both ephemeral and authentication KEM contributions. The KC values are exchanged in `IKE_AUTH` rather than immediately after auth-KEM decapsulation to maintain consistency with IKEv2’s authenticated encryption boundary: all post-authentication payloads (including ESP SA proposals) are protected by the fully derived `Master_Key`.

This design matches the KEMTLS/AuthKEM Finished message structure, where key confirmation occurs after all authentication material has been processed [21, 24, 20].

4.4 Per-Peer Operation Count Comparison

Table 3: Per-peer operation counts per handshake.

Operation	Existing	Proposed-A (ML-KEM auth)	Proposed-B (McEliece auth)
ECDHE KeyGen / agree	1×	1×	1×
KEM KeyGen (ephemeral)	1×	1×	1×
KEM Encap (ephem or auth)	1× (I:0, R:1)	2×	1× ML-KEM + 1× McEliece
KEM Decap	1×	2×	1× ML-KEM + 1× McEliece
DSA Sign	1×	0	0
DSA Verify	1×	0	0
HMAC (key confirmation)	0	1× (negligible)	1× (negligible)

*Counts are per peer for a fully **mutually** authenticated handshake.* In the Existing approach each peer both generates one signature (over its own AUTH payload) and verifies one (the peer's), so the combined both-sides totals are 2 Sign + 2 Verify; the Proposed variants replace these with the per-peer KEM operations shown.

The per-peer figures feed Tables 4 and 5, and Table 6 reports the sum over both peers.

4.5 Backward Compatibility Considerations

The Proposed protocol requires explicit negotiation of KEM-based authentication via the AD-DKE (Additional Key Exchange) mechanism in `IKE_SA_INIT`. Implementations that do not support KEM-Auth will fall back to traditional signature-based authentication. This ensures:

- **Graceful degradation:** Legacy peers continue to operate with ML-DSA or FN-DSA signatures.
- **Negotiation transparency:** The initiator signals KEM-Auth capability in `SA_I`; if the responder does not include KEM-Auth in `SA_R`, the session proceeds with signatures.
- **No protocol breaks:** The 4-exchange structure is a superset of the 3-exchange signature-based flow; additional `IKE_INTERMEDIATE` exchanges are permitted by RFC 9370.

Deployments can adopt the Proposed protocol incrementally without requiring fleet-wide upgrades.

5. Performance Comparison

Security Level III ($\approx$ AES-192); ML-KEM-768 as ephemeral KEM; McEliece figures from libmceliece Golden Cove benchmarks.

The latency tables in this section deliberately exclude the classical ECDHE contribution. Its software cost (ephemeral key generation plus shared-secret agreement) is of the same order of magnitude as a single ML-KEM-768 operation, i.e. small relative to the signature work, and — being identical across all four columns — it cancels in every cross-approach comparison. Readers who need an absolute, end-to-end CPU budget should add the same fixed ECDHE term to each column; doing so does not change any of the relative figures or the qualitative ranking.

Table 4: Table P-1: Initiator latency per handshake (μs , 3.0 GHz Intel x86-64 AVX2). Classical ECDHE cost is omitted as it is identical across all approaches and outside the PQ-specific delta. The shared ephemeral ML-KEM exchange is retained because these tables report the full PQ-handshake CPU budget, not only the auth-specific difference.

Step	Exist.+ML- DSA-65	Exist.+FN- DSA-512	Prop.-A	Prop.-B
KG_ephem (ML-KEM-768)	6	6	6	6
Auth-Encap (to peer's static PK)	—	—	7 (ML-KEM)	18 (McEliece)
Decap_ephem	10	10	10	10
Auth-Decap (from peer)	—	—	10 (ML-KEM)	76 (McEliece)
DSA Sign	45	212	0	0
DSA Verify	24	39	0	0
HMAC (key conf.)	0	0	≈ 1	≈ 1
Total (μs)	85	267	34	111
vs. Exist.+ML-DSA-65	baseline	+214%	-60%	+31%

Table 5: Table P-2: Responder latency per handshake (μs). The shared ephemeral ML-KEM exchange is retained for the same reason as in Table 4.

Step	Exist.+ML- DSA-65	Exist.+FN- DSA-512	Prop.-A	Prop.-B
Encap_ephem (ML-KEM-768)	7	7	7	7
Auth-Decap (from initiator)	—	—	10	76
Auth-Encap (to initiator's static PK)	—	—	7	18
DSA Sign	45	212	0	0
DSA Verify	24	39	0	0
HMAC	0	0	≈ 1	≈ 1
Total (μs)	76	258	25	102
vs. Exist.+ML-DSA-65	baseline	+239%	-67%	+34%

Table 6: Table P-3: Combined handshake CPU time, both peers (μs). The shared ephemeral ML-KEM exchange remains included; only classical ECDHE is omitted.

Metric	Exist.+ML- DSA-65	Exist.+FN- DSA-512	Prop.-A	Prop.-B
Initiator CPU (μs)	85	267	34	111
Responder CPU (μs)	76	258	25	102
Sum both sides	161	525	59	213
vs. Exist.+ML-DSA-65	baseline	+226%	-63%	+32%

Proposed-A replaces two expensive signature operations (sign + verify; $45 + 24 = 69 \mu\text{s}$ per peer for ML-DSA-65) with two KEM Encap/Decap calls ($7 + 10 = 17 \mu\text{s}$ per peer), saving

~60–63%. Using the attached liboqs/Falcon benchmark as the FN-DSA source gives a very different picture: the signature pair costs about **212 + 39 = 251 μ s per peer** for FN-DSA-512 and **436 + 79 = 515 μ s per peer** for FN-DSA-1024 on the common 3.0 GHz normalization. Proposed-B is 32% slower than the ML-DSA-65 baseline in total CPU cost, but its overhead can be justified when wire-efficiency and algorithm diversity are the primary design goals.

Table 7: Table P-4: Existing explicit authentication with two ML-DSA-65 baselines. **Usage guidance:** The optimized AVX2 estimate represents best-case performance with hand-tuned assembly; use this for architectural comparisons and lower-bound projections. The measured liboqs/macOS data represents realistic performance from a widely-deployed library; use this for capacity planning and operational decision-making. The ~70% performance gap reflects the maturity difference between research implementations and production-ready libraries.

Metric	Optimized AVX2 estimate	Measured liboqs/macOS
Initiator CPU (μ s)	85	140.8
Responder CPU (μ s)	76	131.8
Sum both sides (μs)	161	272.7
Signature work per peer (sign + verify)	69	124.8

Both columns correspond to Existing + ML-DSA-65. The left column uses the optimized pq-crystals/SUPERCOP-style estimate; the right column uses the local liboqs benchmark log A.

6. Wire Bytes Comparison

Assumptions: NIST Level III (ML-KEM-768 ephemeral); certificate-based identity includes X.509 wrapper (~200 B); “preloaded” means that only a **20-byte SHA-1 hash** (per RFC 7296 CERTREQ payload conventions) of the peer’s public key is transmitted on the wire. Preloaded certificates are **optional** and not the only deployment mode; identity-privacy implications of each option are discussed in Section 10.5. IKEv2 header overhead (~28 B/message) and encrypted payload framing excluded. All certificate-based rows assume each peer transmits a single end-entity certificate, not a full certificate chain; including one or more intermediate-CA certificates would add their encodings to the wire cost of every certificate-based approach, explicit and implicit alike (see the note following Figure 1).

Table 8: Table W-1 (a): Existing + ML-DSA-65 (certificates).

Message	Contents	B
IKE_INT. (I→R)	ML-KEM-768 ephem PK	1 184
IKE_INT. (R→I)	ML-KEM-768 ephem CT	1 088
IKE_AUTH (I→R)	ML-DSA-65 cert (pk 1 952 + ~200) + Sig (3 309)	5 461
IKE_AUTH (R→I)	Same	5 461
Total		13 194

Table 9: Table W-1 (a-pre): Existing + ML-DSA-65 (preloaded certificates).

Message	Contents	B
IKE_INT. (I→R)	ML-KEM-768 ephem PK	1 184
IKE_INT. (R→I)	ML-KEM-768 ephem CT	1 088
IKE_AUTH (I→R)	SHA-1 fingerprint (20) + ML-DSA-65 Sig (3 309)	3 329
IKE_AUTH (R→I)	Same	3 329
Total		8 930

Table 10: Table W-1 (b): Existing + FN-DSA-512 (certificates).

Message	Contents	B
IKE_INT. (I→R)	ML-KEM-768 ephem PK	1 184
IKE_INT. (R→I)	ML-KEM-768 ephem CT	1 088
IKE_AUTH (I→R)	FN-DSA-512 cert (pk 897 + ~200) + Sig (666)	1 763
IKE_AUTH (R→I)	Same	1 763
Total		5 798

Note (FN-DSA-512 preloaded). For symmetry with the ML-DSA-65 case, the corresponding preloaded-certificate variant of Existing + FN-DSA-512 would total $1,184 + 1,088 + 2 \times (20 + 666) = 3,644$ B, computed exactly as in Table 9. It is omitted from the headline summary tables only to keep them focused: FN-DSA’s distinguishing advantage is its compact on-wire signature, so the certificate-bearing row (Table 10) is retained as its representative case. The 3,644 B figure is noted here for completeness.

Table 11: Table W-1 (c-cert): Proposed-A — ML-KEM-768 auth, on-wire certificates.

Message	Contents	B
IKE_INT. (ephem, I→R)	ML-KEM-768 ephem PK (1 184)	1 184
IKE_INT. (ephem, R→I)	ML-KEM-768 ephem CT (1 088) + Cert_R (ML-KEM-768 pk 1 184 + ~200 wrapper)	2 472
IKE_INT. (auth, I→R)	Cert_I (1 184 + ~200) + Auth CT_I (1 088)	2 472
IKE_INT. (auth, R→I)	Auth CT_R (1 088)	1 088
IKE_AUTH (I→R)	IDI + KC_I (48) + ESP SA proposals	~120
IKE_AUTH (R→I)	IDr + KC_R (48) + ESP SA response	~120
Total		~7 456

Note. Table 11 provides the derivation for the “Proposed-A (ML-KEM auth, certs)” row of the summary Table 14: relative to the preloaded variant (Table 12), each peer transmits a full

KEM certificate ($\sim 1,384$ B) in place of a 20-byte fingerprint, adding $2 \times (1,384 - 20) = 2,728$ B, i.e. $4,728 + 2,728 = 7,456$ B.

Table 12: Table W-1 (c): Proposed-A — ML-KEM-768 auth, preloaded keys.

Message	Contents	B
IKE_INT. (ephem, I→R)	ML-KEM-768 ephem PK (1 184)	1 184
IKE_INT. (ephem, R→I)	ML-KEM-768 ephem CT (1 088) + SHA-1 fingerprint (20)	1 108
IKE_INT. (auth, I→R)	SHA-1 fingerprint (20) + Auth CT _I (1 088)	1 108
IKE_INT. (auth, R→I)	Auth CT _R (1 088)	1 088
IKE_AUTH (I→R)	IDi + KC _I (48) + ESP SA proposals	~ 120
IKE_AUTH (R→I)	IDr + KC _R (48) + ESP SA response	~ 120
Total		$\sim 4\,728$

Note: The responder’s certificate (or SHA-1 fingerprint) is transmitted in the first IKE_INTERMEDIATE (ephemeral KE exchange), not the second (auth-KEM exchange), so the initiator knows which static public key to use for CT_{auth,I}. The initiator’s certificate/fingerprint is transmitted in the second IKE_INTERMEDIATE along with CT_{auth,I}.

Table 13: Table W-1 (d): Proposed-B — McEliece-460896 auth, preloaded keys.

Message	Contents	B
IKE_INT. (ephem, I→R)	ML-KEM-768 ephem PK (1 184)	1 184
IKE_INT. (ephem, R→I)	ML-KEM-768 ephem CT (1 088) + SHA-1 fingerprint (20)	1 108
IKE_INT. (auth, I→R)	SHA-1 fingerprint (20) + McEliece CT _I (156)	176
IKE_INT. (auth, R→I)	McEliece CT _R (156)	156
IKE_AUTH (I→R)	IDi + KC _I (48) + ESP SA	~ 120
IKE_AUTH (R→I)	IDr + KC _R (48) + ESP SA	~ 120
Total		$\sim 2\,864$

Note: The responder’s certificate (or SHA-1 fingerprint) is transmitted in the first IKE_INTERMEDIATE (ephemeral KE exchange), not the second (auth-KEM exchange), so the initiator knows which static public key to use for CT_{auth,I}. The initiator’s certificate/fingerprint is transmitted in the second IKE_INTERMEDIATE along with CT_{auth,I}.

Table 14: Table W-2: Wire bytes summary and reduction (handshake payload model; IKEv2 headers and framing excluded).

Approach	Total (B)	vs. ML-DSA-65	vs. FN-DSA-512
Existing + ML-DSA-65 (certs)	13 194	baseline	+128%
Existing + ML-DSA-65 (preloaded)	8 930	-32%	+54%
Existing + FN-DSA-512 (certs)	5 798	-56%	baseline
Proposed-A (ML-KEM auth, certs)	~7 456	-44%	+29%
Proposed-A (ML-KEM auth, preloaded)	~4 728	-64%	-18%
Proposed-B (McEliece, preloaded)	~2 864	-78%	-51%

The McEliece-460896 public key (524 KB) is preloaded and never transmitted; only a 20-byte SHA-1 fingerprint (per RFC 7296 CERTREQ conventions) is sent per session. The McEliece ciphertext is 156 B versus ML-KEM-768’s 1 088 B — a $7\times$ reduction. Preloaded certificates are an optional deployment mode; see Section 10.5 for identity-privacy considerations when certificates are transmitted on the wire.

What the approximate totals include: The $\sim$ rows are approximate only because each `IKE_AUTH` message is modeled as ~ 120 B for ID payloads, KC, and ESP SA negotiation; exact size varies with ID length, ESP transform lists, and optional notifies. The preloaded totals therefore still include the full common ML-KEM ephemeral exchange (1,184 B PK + 1,088 B CT = 2,272 B), both auth-KEM ciphertexts ($2\times 1,088$ B for Proposed-A; 2×156 B for Proposed-B), two 20-byte certificate fingerprints, and the two `IKE_AUTH` messages. They are not just the SHA-1 fingerprints.

Fairness note: Preloading certificates reduces wire bytes for *all* approaches, not just KEM-based authentication. The Existing + ML-DSA-65 (preloaded) row shows that signature-based authentication also benefits significantly from preloaded certificates (32% reduction). With on-wire certificates, Proposed-A still beats Existing + ML-DSA-65 on wire bytes, but FN-DSA-512 remains more compact; with preloaded certificates, Proposed-A again becomes the smaller of the two practical alternatives.

Certificate Revocation Considerations. When using preloaded KEM certificates (20-byte SHA-1 fingerprints), certificate revocation checking becomes an operational consideration:

- **Traditional PKI:** If certificates are transmitted on the wire, standard CRL or OCSP mechanisms apply. The responder can check revocation status before performing auth-KEM decapsulation.
- **Preloaded keys:** Out-of-band distribution bypasses traditional PKI revocation. Recommended approaches include:
 - Maintain a local revocation list keyed by SHA-1 fingerprint.
 - Use short-lived certificates (e.g., 24-hour validity) to limit revocation window.
 - Implement emergency key rollover procedures for compromised keys.
- **McEliece-specific:** The large public key size (524 KB for McEliece-460896) makes frequent key rotation expensive. Deployments using Proposed-B should plan for longer key lifetimes (e.g., 1–3 years) with robust revocation infrastructure.

7. Implementation Memory Requirements

Table 15: Table M-1: Code size, compiled binary, optimized reference C. Source: Halak et al. 2025 [16]; x86 estimates scaled from ARM RP2040 measurements.

Algorithm	Code (KB)	Notes
ML-KEM-768	~6	RP2040 study: 5.1–6.7 KB
ML-KEM-1024	~7	RP2040 study
ML-DSA-65	~9	RP2040 study: 8.4–8.9 KB
FN-DSA-512	~20–30	FFT-based Gaussian sampler
McEliece-348864	~50–80	Binary field arithmetic, Goppa decode
McEliece-6688128	~100–150	Same structure, larger parameters
Shared utils	~5	SHAKE/Keccak (~3 KB), poly (~2 KB)

Table 16: Table M-2: Peak dynamic memory per operation.

Algorithm / Operation	Peak RAM (KB)	Dominant cost
ML-KEM-768 KeyGen	~8	Public key + working polynomials
ML-KEM-768 Encap	~6	CT + shared secret + buffer
ML-KEM-768 Decap	~7	Same + private key (2.4 KB)
ML-DSA-65 Sign	~20–25	Poly matrices, rejection sampling
ML-DSA-65 Verify	~15	Matrix expansion, poly temporaries
FN-DSA-512 Sign	~30–50	Gaussian sampler tree (deep stack)
FN-DSA-512 Verify	~5	Simple polynomial check
McEliece-460896 Encap	~530	Public key in RAM (524 KB)
McEliece-460896 Decap	~30	Private key (13.6 KB) + syndrome decode
McEliece-6688128 Encap	~1 050	Public key (1 020 KB)
McEliece-6688128 Decap	~30	Small private key only

Table 17: Table M-3: Total implementation memory footprint (handshake-time).

Approach	Code	Peak Dyn.	Total
Existing: ML-KEM-768 + ML-DSA-65	~20 KB	~30 KB	~ 50 KB
Existing: ML-KEM-768 + FN-DSA-512	~37 KB	~55 KB	~ 92 KB
Proposed-A: ML-KEM-768 (both)	~11 KB	~22 KB	~33 KB
Proposed-B: ML-KEM-768 + McEliece-460896 (pk mmapped)	~90 KB	~550 KB	~ 640 KB
Proposed-B: McEliece-6688128 (pk mmapped)	~115 KB	~32 KB	~ 147 KB

For Proposed-B at NIST Level III, the encapsulating party must hold or memory-map the McEliece-460896 public key (524 KB). On VPN gateways with GB-class RAM this is trivial. The decapsulating party needs only the ~14 KB private key.

8. Protocol Exchange Count Comparison

8.1 Analysis

An earlier version of this document claimed a 1.5-exchange flow with preloaded keys. This is not achievable in a backward-compatible IKEv2 deployment.

Combining ephemeral KE and auth-KEM in a single `IKE_INTERMEDIATE` is permitted by RFC 9370 with fragmentation, but requires explicit allowance in the KEM-Auth draft.

8.2 Exchange Count (1 exchange = 1 RT)

Throughout this section, an IKEv2 *exchange* means one request/response pair, i.e., one network round trip (RT). The two counts are therefore identical here, and the tables below use **exchange count** as the primary term.

Existing (PQ signatures + hybrid KE via RFC 9370):

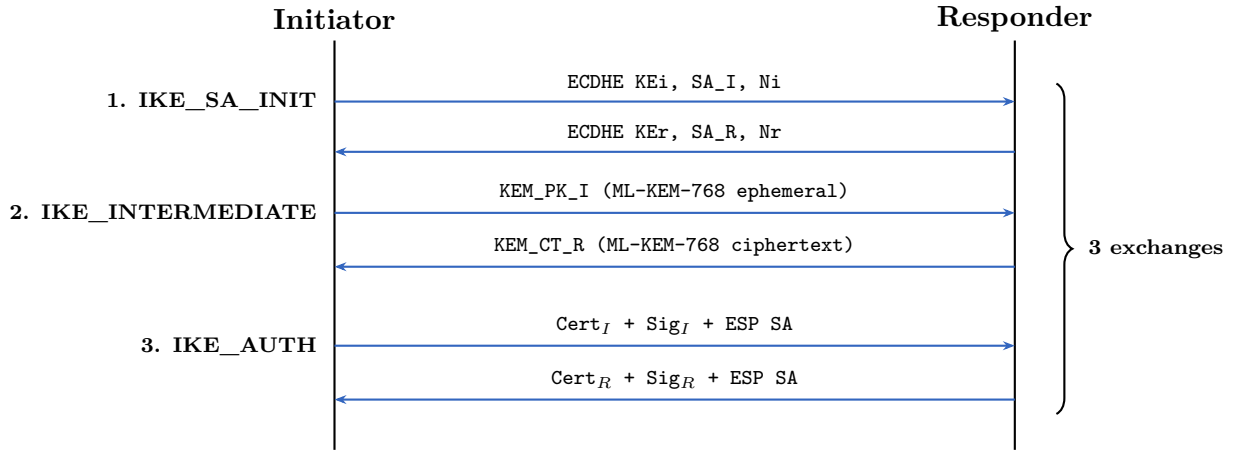


Figure 3: Existing (hybrid ECDHE + PQ signatures): 3 exchanges. Classical ECDHE remains in `IKE_SA_INIT`; ML-KEM-768 ephemeral KE is carried in `IKE_INTERMEDIATE`; certificate, signature, and ESP SA setup remain in `IKE_AUTH`.

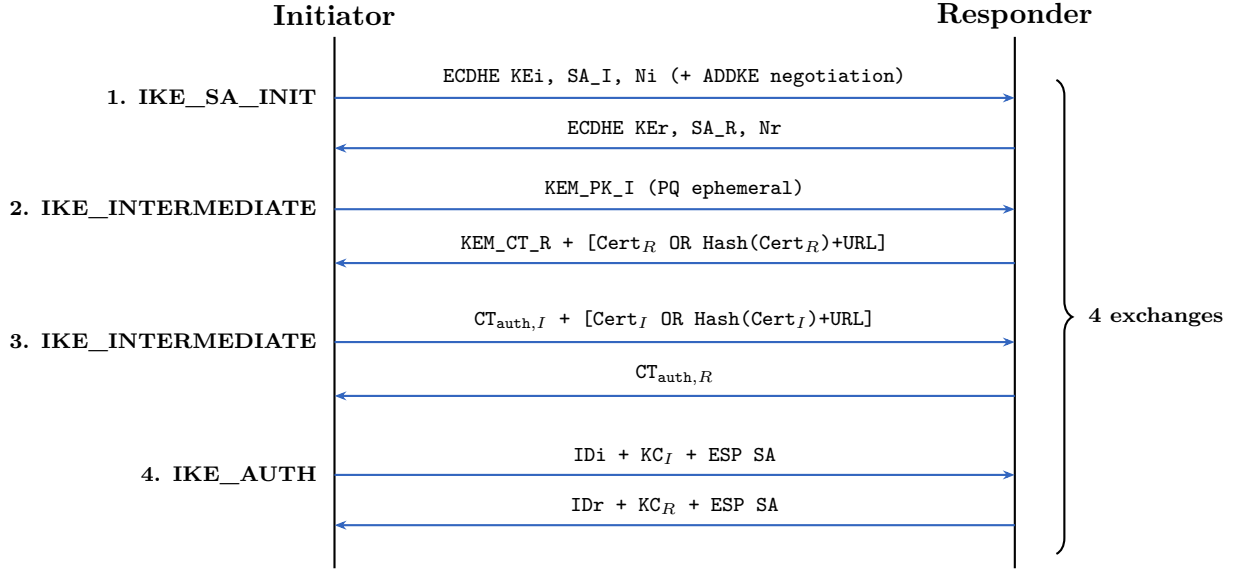
Proposed (Implicit KEM-Auth, as currently drafted):

Figure 4: Proposed (implicit KEM-Auth, as drafted): 4 exchanges. One extra `IKE_INTERMEDIATE` carries the auth-KEM payloads, while `IKE_AUTH` carries both KC values and ESP SA negotiation. **Cert_R placement:** The responder transmits its certificate (or 20-byte SHA-1 hash + URL) together with `KEM_CT_R` in the *first* `IKE_INTERMEDIATE` response, so the initiator knows `PKR,static` when computing `CTauth,I` in the next exchange. The initiator returns its own certificate (or hash) in the *second* `IKE_INTERMEDIATE` request alongside `CTauth,I`; the responder's reply in that exchange carries only `CTauth,R`. **Note on identity payloads:** Per RFC 7296, IKEv2 Identity Payloads (IDi/IDr) appear only in `IKE_AUTH`; they are not transmitted as separate payloads in `IKE_INTERMEDIATE`. Identity information reaches the peer at the `IKE_INTERMEDIATE` stage only *indirectly*, through the contents of any KEM certificate that is exchanged on the wire.

Table 18: Table R-1: Exchange count (1 exchange = 1 RT).

Approach	Exchanges	Notes
Existing (PQ KE per RFC 9370 + PQ sig)	3	<code>IKE_SA_INIT</code> + <code>IKE_INTERMEDIATE</code> + <code>IKE_AUTH</code>
Proposed (implicit KEM-Auth, as drafted)	4	Additional <code>IKE_INTERMEDIATE</code> for auth KEM; KC values stay in <code>IKE_AUTH</code>
Variant (auth KEM combined with ephemeral KE)	3	Fewer exchanges, but identity privacy is weakened

The Proposed approach, as currently specified, requires **one additional exchange** relative to the existing PQ-signature deployment. On a 20 ms per-RT link this adds 20 ms, partially offsetting the CPU savings of Section 5. This latency trade-off is consistent with the broader KEM-authentication literature: Muckle# reports that KEM-based authentication can reduce bandwidth and CPU costs while requiring additional communication rounds relative to signature-based Muckle+ [34].

Security cost of combining the KEM-Auth exchange with ephemeral KE

A tempting optimization is to carry $CT_{auth,I}$ and $CT_{auth,R}$ in the same `IKE_INTERMEDIATE` as the PQ ephemeral KEM PK/CT, reducing the exchange count from 4 to 3. However, this optimization carries a meaningful security cost.

The full analysis is in Section 10.5. **The current 4-exchange structure is the correct design — collapsing it requires explicitly accepting the identity-privacy downgrade.**

9. Summary Comparison Tables

Table 19: Table S-1: Computational performance (μ s, both peers combined).

Approach	CPU	Relative	Recommendations
Existing + ML-DSA-65 (optimized AVX2 estimate)	161	1.00×	Current IETF default
Existing + ML-DSA-65 (measured liboqs/macOS)	273	1.70×	Concrete library-build data point
Existing + FN-DSA-512	525	3.26×	Compact on wire, slower in generic software
Proposed-A (ML-KEM-768)	59	0.37×	Best simplicity/speed
Proposed-B (McEliece-460896)	213	1.32×	Algorithm diversity

Table 20: Table S-2: Bytes over the wire (handshake payload model; IKEv2 headers and framing excluded).

Approach	Bytes	Relative	Dominant Cost
Existing + ML-DSA-65 (certs)	13 194	1.00×	ML-DSA-65 pk+sig ×2
Existing + ML-DSA-65 (preloaded)	8 930	0.68×	ML-DSA-65 sig ×2
Existing + FN-DSA-512 (certs)	5 798	0.44×	FN-DSA sig (666 B)
Proposed-A (ML-KEM auth, certs)	~7 456	0.56×	2 KEM certs + 2 auth CTs
Proposed-A (ML-KEM auth, preloaded)	~4 728	0.36×	Ephem ML-KEM + 2 auth CTs
Proposed-B (McEliece, preloaded)	~2 864	0.22×	Ephem ML-KEM dominates

Note: The $\sim$ rows are approximate only because the two `IKE_AUTH` messages are modeled as ~ 120 B each for ID payloads, KC, and ESP SA negotiation. The preloaded totals still include the common ML-KEM-768 ephemeral exchange (2,272 B total), both auth-KEM ciphertexts, and two 20-byte certificate fingerprints.

Table 21: Table S-3: Implementation memory requirements.

Approach	Code	Peak Dyn.	Complexity
Existing + ML-DSA-65	~20 KB	~30 KB	Moderate
Existing + FN-DSA-512	~37 KB	~55 KB	High (Gaussian sampler)
Proposed-A (ML-KEM-768)	~11 KB	~22 KB	Lowest
Proposed-B (McEliece, pk mmapped)	~115 KB	~32 KB	Moderate
Proposed-B (McEliece, pk in RAM)	~115 KB	~550–1,050 KB	Hardware-dependent

Table 22: Table S-4: Exchange count (1 exchange = 1 RT).

Approach	Exchanges	Notes
Existing (Hybrid KE per RFC 9370 + PQ sig)	3	60 ms at 20 ms per RT
Proposed (implicit KEM-Auth, as drafted)	4	80 ms at 20 ms per RT
Variant (auth KEM + ephem KE combined)	3	60 ms; not recommended — see Section 10.5

Net assessment: Proposed-A is better on CPU (-63%), code size, peak RAM, and implementation simplicity. Proposed-B is better on wire bytes (-78% in the preloaded deployment model) and on algorithm diversity, at the cost of higher code size and modest CPU overhead. Both proposed variants keep the same exchange count, and both require one additional exchange relative to the existing signature-based flow. **Overall winner: Proposed-A**, because it offers the best overall balance of speed, footprint, and deployability while preserving the current draft’s identity-privacy properties. Proposed-B remains the better fit when minimizing wire bytes or avoiding single-family lattice dependence is the overriding requirement.

10. Security Analysis

Note: The first formal security model of IKEv2 in a quantum setting (Tamarin Prover), demonstrating that the standard IKEv2 protocol becomes vulnerable to quantum adversaries and that hybrid extensions restore the intended security properties, has been provided in [6].

10.1 Explicit Authentication Security

Digital signatures in IKEv2 provide entity authentication, non-repudiation, and require no separate key confirmation. The existing IETF approach is straightforward but expensive: each handshake invokes one signature generation and one verification per peer, dominating the CPU budget.

Effect of PPK. Adding a post-quantum preshared symmetric key (PPK) which is shared among multiple parties does not change the explicit-authentication semantics: signatures still provide peer authentication and non-repudiation, and signature verification still gates acceptance of the peer. What PPK changes is the *confidentiality margin* of the protected handshake

and Child-SA keys. Once mixed into the IKEv2 key schedule, the traffic keys no longer depend only on ECDHE and the PQ exchange; a passive recorder must also recover the PPK. PPK therefore strengthens resistance to store-now-decrypt-later attacks, but it does not replace signatures, does not reduce signature cost, and does not add non-repudiation beyond what signatures already provide.

Quantum Security Level. The hybrid construction provides security against quantum adversaries as follows:

- **Session key confidentiality:** Protected by ML-KEM-768 (NIST Security Level III, equivalent to AES-192 against quantum adversaries [7]). A CRQC can break ECDHE but cannot break ML-KEM-768 without solving Module-LWE in dimension 768 [7].
- **Authentication (Proposed-A):** Protected by ML-KEM-768 auth KEM. A CRQC cannot forge auth-KEM ciphertexts without breaking IND-CCA2 security of ML-KEM-768 [7].
- **Authentication (Proposed-B):** Protected by McEliece-460896 (NIST Security Level III via syndrome decoding [11]). Provides algorithm diversity: a break in lattice-based cryptography does not compromise authentication.
- **Forward secrecy:** Ephemeral ML-KEM-768 ensures that compromise of static keys does not compromise past session keys [7].

The classical ECDHE component provides **no security** against a CRQC but ensures that the protocol remains secure against classical adversaries even if a catastrophic break in ML-KEM is discovered before deployment [30, 4].

10.2 Implicit Authentication — MQV/HMQV Lineage

MQV [19]: Authenticated Diffie-Hellman without signatures, using static and ephemeral key pairs; implicit authentication derived from the combined shared secret.

HMQV [18] (Krawczyk, CRYPTO 2005): Hardened MQV with tight security reductions under the CDH assumption in the Random Oracle Model, defending against UKS (unknown key-share), KCI (key-compromise impersonation), and reflection attacks.

KEMTLS [21] (Stebila, Schwabe, Wiggers, CCS 2020): First formal treatment of KEM-based implicit authentication for TLS 1.3. Removes handshake signatures entirely; formal security proof; Tamarin verification [23].

KEMTLS-PDK [22] (ESORICS 2021): Extension to pre-distributed public keys; analogous to the preloaded-key variant in Section 6.

PQuAKE [20] (Blumenthal et al., draft-00, April 2025): Applies dual-KEM implicit authentication to a general authenticated key exchange setting; direct conceptual predecessor to the IKEv2 KEM-Auth design. Based on an unpublished-yet manuscript on compact post-quantum authenticated key exchange with identity hiding; provides the closest direct antecedent for the dual-KEM operational structure, key confirmation split, and implicit-rejection discussion carried into this IKEv2 adaptation; includes formal proofs of the protocol.

AuthKEM [24] (Wiggers et al., draft-06, November 2025): IETF standardization of KEMTLS for TLS 1.3; specifies full and abbreviated handshakes with unilateral and mutual authentication.

Muckle# [34] (Battarbee et al., EPJ Quantum Technology, 2025; earlier circulated as IACR ePrint 2025/1228): Extends KEM-based authentication to a hybrid AKE framework that mixes conventional, post-quantum, and QKD-derived key material. It is not an IKEv2 design and does not resolve the RFC 7296 identity-privacy sequencing issue analyzed in Section 10.5, but it is public prior art for KEM-based authentication inside a hybrid key schedule.

Security properties of the Proposed protocol:

- **Forward secrecy** [18, 21, 20] — ephemeral KEM ensures past sessions are safe even if static keys leak.
- **KCI resistance** [18] — KCI (key-compromise impersonation) resistance means that learning one party’s own static secret key does not let the attacker impersonate the other peer to that compromised party.
- **UKS resistance** [18] — UKS (unknown key-share) resistance means that an attacker cannot cause two honest peers to derive the same key while disagreeing about with whom the key is shared; binding both identifiers into the derivation prevents this.
- **Mutual authentication** [21, 20] — each peer must decapsulate a ciphertext encrypted to its own static public key to participate.

Security reduces to the IND-CCA2 security of both the authentication KEM and the ephemeral KEM [21, 5]. The 48-byte Key Confirmation (HMAC) is the direct analog of the KEMTLS/AuthKEM Finished messages [21, 24, 20] and ensures both parties derived the same master key before traffic begins.

Effect of PPK. In the implicit-authentication design, PPK has two effects. First, like in the explicit-signature case, it adds an independent symmetric secret to the final key schedule, so recovering ECDHE and even one PQ component is still insufficient without the PPK. Second, because RFC 9867 mixes the PPK in before the auth-KEM exchange is processed, it strengthens confidentiality of the auth-KEM ciphertexts, identity payloads, and long-term certificate material carried in subsequent encrypted exchanges. PPK therefore materially improves passive-adversary resilience and identity protection, but it does not by itself authenticate either peer and it does not remove the need for auth-KEM decapsulation or KC verification.

10.3 Non-Repudiation Trade-off

Implicit authentication does *not* provide non-repudiation [18]. For IKEv2/IPsec session establishment this is generally acceptable, and often desirable. If non-repudiation is required, explicit signatures must be retained.

10.4 McEliece-Specific Security Diversity (Proposed-B)

Classic McEliece [11] is based on the code-based hardness assumption (syndrome decoding / binary Goppa code indistinguishability [33, 11]), a completely different mathematical family from ML-KEM’s Module-LWE [7]. A structural break in lattice-based cryptography cannot compromise sessions protected by Proposed-B without simultaneously solving a ~50-year open problem in algebraic coding theory [33].

That said, the broader idea of exploiting **Classic McEliece specifically in the authentication role** when certificates or public keys are cached/pre-distributed is not new: **KEMTLS-**

PDK [22] identifies Classic McEliece as suitable for authentication in exactly that deployment model, because the large public key need not be retransmitted while the ciphertext stays compact. Proposed-B here is therefore best viewed as an **IKEv2-specific adaptation and quantitative evaluation** of that broader KEMTLS-PDK insight, rather than as a wholly novel cryptographic direction.

10.5 Identity Privacy and the Cost of Combining Exchanges

Why the current 4-exchange design protects identities

In the as-drafted Proposed protocol, the KEM-Auth `IKE_INTERMEDIATE` exchange is sent after both the classical ECDHE exchange (`IKE_SA_INIT`) and the PQ ephemeral KE exchange (first `IKE_INTERMEDIATE`) have completed. At that point the channel is protected by the combined keying material of:

1. Classical ECDHE shared secret (SS_{cl});
2. PQ ephemeral KEM shared secret (SS_1);
3. PPK, if negotiated (RFC 9867, applied at `IKE_INTERMEDIATE` stage).

An adversary recording traffic must break *all three* protections simultaneously to recover the auth-KEM ciphertexts ($CT_{auth,I}$, $CT_{auth,R}$) or the identity payloads (ID_i , ID_r). A cryptographically relevant quantum computer (CRQC) can break the ECDHE component via Shor’s algorithm, but cannot break a correctly implemented PQ KEM or a high-entropy PPK. Authentication data is therefore protected against passive quantum adversaries under the current design.

Security cost of combining the KEM-Auth exchange with ephemeral KE. A tempting optimization is to carry $CT_{auth,I}$ and $CT_{auth,R}$ in the same `IKE_INTERMEDIATE` as the PQ ephemeral KEM PK/CT, reducing the exchange count from 4 to 3. However, this optimization carries a meaningful security cost.

At the time the combined `IKE_INTERMEDIATE` is sent, only the classical ECDHE shared secret (SS_{cl}) is available as channel protection — the PQ KEM shared secret (SS_1) has not yet been established. As discussed in 8.2, a passive adversary with a CRQC can break ECDHE offline, and thereby decrypt the combined `IKE_INTERMEDIATE`, recovering, as listed in 8.2:

- The auth-KEM ciphertexts ($CT_{auth,I}$, $CT_{auth,R}$);
- The identity payloads ID_i and ID_r (if present in the same message);
- The content of KEM certificates, if certificates are transmitted rather than pre-distributed fingerprints.

This violates the **IKEv2 identity-privacy guarantee**: RFC 7296 [1] specifies that both peers’ identities must be protected against passive adversaries. The current 4-exchange design upholds that guarantee against a quantum adversary through the additional protection of SS_1 (and optionally the PPK per RFC 9867 [29]). The combined 3-exchange variant does not, and the optimization must not be adopted without an explicit downgrade of the identity-privacy claim.

Identity privacy with preloaded certificates. Preloaded certificates reduce what is transmitted on the wire: instead of a full certificate, only a 20-byte SHA-1 fingerprint (per RFC 7296 CERTREQ conventions) is sent. This is a meaningful reduction, but it does not eliminate the identity-privacy risk in the combined-exchange scenario:

- A CRQC-capable adversary who decrypts the combined `IKE_INTERMEDIATE` can still recover the SHA-1 fingerprints and use them to **identify and track** specific peers across sessions, even without learning full certificate content.
- Even in the current 4-exchange design, if PPK is not negotiated, the auth-KEM data is protected only by SS_1 ; the CRQC threat remains an inherent property of the PQ KEM's forward-secrecy boundary for that layer alone.

Preloaded certificates are an optional deployment mode. In deployments where certificates are transmitted on the wire (the general case), the identity-privacy analysis above applies in full, and the 4-exchange structure of the current draft must be preserved to maintain the IKEv2 identity-privacy guarantee against quantum-capable passive adversaries.

Recommendation. Deployments requiring maximum identity privacy against quantum-capable passive adversaries **SHOULD**:

- Use the 4-exchange structure (do not combine auth-KEM with ephemeral KE);
- Negotiate PPK per RFC 9867 to provide triple-layer quantum defense;
- Use preloaded certificates to minimize on-wire fingerprint exposure.

Active Adversary Considerations

While the 4-exchange structure protects identity privacy against passive quantum adversaries, active adversaries present different risks:

- **Man-in-the-middle (MITM):** An active adversary can always learn the identity of the first party to send their identity, by observing which static keys are used for auth-KEM. This is inherent to any certificate-based authentication system and is not specific to KEM-Auth.
- **Denial-of-service:** An attacker can cause auth-KEM decapsulation failures by corrupting CT_{auth} payloads, forcing session establishment to fail. This is equivalent to corrupting signatures in the existing approach.
- **Identity probing:** If preloaded keys are used, an attacker can probe which peers a responder has preloaded keys for by sending different SHA-1 fingerprints and observing success/failure.

The Proposed protocol provides the same active-adversary resistance as signature-based authentication: mutual authentication prevents MITM, but does not hide identities from an active attacker who can observe or manipulate the handshake.

10.6 Failure Modes and Error Handling

Auth-KEM Decapsulation Failure. If either peer fails to decapsulate the auth-KEM ciphertext ($CT_{auth,I}$ or $CT_{auth,R}$), the session **MUST** be aborted. Possible causes include:

- **Wrong static key:** The encapsulating peer used an incorrect or outdated public key.
- **Corrupted ciphertext:** Network transmission error or active attack.
- **Implementation error:** Incorrect KEM algorithm negotiation.

Unlike signature verification failure (which is unambiguous), KEM decapsulation failure does not distinguish between wrong key and corrupted data [7, 20] (ML-KEM uses implicit rejection: a decapsulation failure returns a pseudorandom value indistinguishable from a valid shared secret, and the mismatch is detected only when Key Confirmation fails). Implementations SHOULD log the failure with sufficient detail for operational diagnosis while avoiding information leakage to potential attackers.

Key Confirmation Failure. If KC_I or KC_R verification fails, the session MUST be aborted. This indicates that the two peers derived different `Master_Key` values, which can occur if:

- One peer used an incorrect static key for auth-KEM encapsulation.
- The key schedule implementation differs between peers.
- An active attacker modified auth-KEM ciphertexts in transit.

Key confirmation failure provides explicit cryptographic evidence of authentication failure, closing the implicit authentication gap [20].

11. Conclusions

This document has compared two approaches to post-quantum mutual authentication in IKEv2 across four dimensions: computational cost, bytes on the wire, implementation memory, and protocol exchange count. The results consistently favour the Proposed-Implicit (Dual-KEM) approach for general deployment, and strongly favour Proposed-B for environments that require algorithm diversity.

Performance Summary

CPU cost (both peers combined, NIST Level III): Replacing the ML-DSA-65 signature pair (sign + verify, $\sim 69 \mu s$ per peer) with two ML-KEM-768 Encap/Decap calls ($\sim 17 \mu s$ per peer) reduces total cryptographic computation by **63%** — from $161 \mu s$ to $59 \mu s$. Using the attached liboqs/Falcon benchmark as the FN-DSA source, the FN-DSA-512 alternative lands at **525 μs** combined CPU time, so Proposed-A is materially faster as well as simpler to implement. Proposed-B runs at $213 \mu s$ — 32% above the ML-DSA-65 baseline — but is well within practical bounds for VPN gateways.

Bytes over the wire: The ML-DSA-65 public key (1 952 B) and signature (3 309 B), sent by each peer, account for over 10 KB of the 13 194 B traditional handshake. Replacing them with KEM ciphertexts reduces total wire bytes by **44%** (Proposed-A with on-wire KEM certificates: $\sim 7\,456$ B), by **64%** (Proposed-A with preloaded certificates: $\sim 4\,728$ B), or by **78%** (Proposed-B, McEliece preloaded: $\sim 2\,864$ B). Preloading certificates also benefits signature-based authentication (reducing Existing + ML-DSA-65 from 13 194 B to 8 930 B, a 32% reduction), but the KEM-based approaches achieve greater reductions due to smaller ciphertext sizes. The McEliece ciphertext is only 156 B versus 1 088 B for ML-KEM — a $7\times$ reduction — because the large McEliece public key is distributed out-of-band and only a 20-byte SHA-1

fingerprint (per RFC 7296) is sent on the wire.

Implementation footprint: Proposed-A requires only a single KEM library. Code size drops from ~ 20 KB (ML-DSA-65 approach) to ~ 11 KB; peak dynamic RAM from ~ 30 KB to ~ 22 KB. There is no Gaussian sampler, no rejection-sampling variance, and no floating-point requirement — all significant pain points in FN-DSA implementations.

Protocol exchange count: As currently specified in draft-wang-ipsecme-kem-auth-ikev2-03, the Proposed approach requires 4 exchanges (vs. 3 for the existing PQ-signature approach), placing both key-confirmation MACs (KC_I and KC_R) in `IKE_AUTH` and auth-KEM ciphertexts in a dedicated `IKE_INTERMEDIATE`. This design is intentional: it ensures that authentication data and peer identities are protected not only by classical ECDHE but also by the PQ ephemeral KEM shared secret, maintaining IKEv2’s identity-privacy guarantee against a passive quantum adversary. Collapsing the auth-KEM exchange into the ephemeral-KE `IKE_INTERMEDIATE` would save one exchange, but expose identity material to CRQC-capable passive adversaries and is therefore not recommended (see Section 10.5).

Algorithm Agility and Future-Proofing

The Proposed protocol supports algorithm agility through standard IKEv2 negotiation mechanisms:

- **Ephemeral KEM:** Negotiated in `IKE_SA_INIT` via `ADDKE`. Peers can offer ML-KEM-512, ML-KEM-768, or ML-KEM-1024 and negotiate the strongest mutually supported parameter set.
- **Auth-KEM:** Determined by the static key embedded in the certificate. Transitioning from ML-KEM-768 to ML-KEM-1024 requires certificate reissuance but does not require protocol changes.
- **Hybrid auth-KEM:** A future extension could support multiple auth-KEM algorithms in parallel (e.g., ML-KEM-768 + McEliece-460896 both used for authentication), with `Master_Key` derived from all shared secrets. This would require a protocol extension to carry multiple `CT_auth` payloads. Muckle# [34] is relevant prior public work because it demonstrates KEM-based authentication inside a hybrid AKE/HAKE key schedule; the remaining work here is the IKEv2-specific negotiation, payload, and failure semantics for multiple authentication KEMs.

The dual-KEM authentication framework is algorithm-agnostic and can accommodate future NIST-approved KEMs without structural protocol changes; extensions that use multiple authentication KEMs should be specified explicitly so that transcript binding, downgrade protection, and failure semantics remain unambiguous.

Overall Assessment

The Proposed-Implicit approach is a sound, well-precedented design for incorporating post-quantum cryptography into IKEv2. Its security foundations are solid: the dual-KEM construction descends directly from the MQV/HMQV family of authenticated key exchange protocols, and its security reduces to the IND-CCA2 hardness of the underlying KEMs. The Key Confirmation exchange (48-byte HMAC, both values exchanged in `IKE_AUTH`) closes the one gap implicit authentication leaves, and the approach has been formally verified in the analogous TLS setting (KEMTLS, Tamarin [23], ESORICS 2022).

The principal advantages over explicit-signature authentication are not marginal: a 63% CPU reduction, a 44–64% bandwidth reduction for Proposed-A depending on certificate handling, and a 78% reduction for Proposed-B are material for high-throughput VPN gateways and bandwidth-constrained links alike. The implementation is simpler — one algorithm family instead of two, no complex Gaussian samplers — and the Proposed-B variant delivers code-based algorithm diversity natively within the key schedule, without the bandwidth penalty of a separate classical hybrid.

The sole operational trade-off is the absence of non-repudiation, which is immaterial for IKEv2 IPsec session establishment. The extra exchange relative to the signature-based approach is not a defect but a necessary structural feature that preserves IKEv2’s identity-privacy guarantee against quantum-capable passive adversaries. The 4-exchange structure must be preserved.

We conclude that the Proposed-Implicit Dual-KEM approach, as specified in draft-wang-ipsecme-kem-auth-ikev2-03, is a secure, efficient, and lean to implement method for post-quantum authentication in IKEv2. It is also directly applicable to other key-exchange protocols that carry authentication in a dedicated message exchange. The parallel standardization of the same design in AuthKEM (draft-celi-wiggers-tls-authkem) for TLS 1.3 and the Muckle# HAKE construction [34] demonstrate broad community confidence that KEM-based authentication is a reusable design pattern rather than an IKEv2-specific anomaly.

Acknowledgments

We are grateful to Brandon Luo, Valery Smyslov (especially — for a lot of recommendations, and his contributions to PPK [29]), and Guilin Wang for collaboration on this work, and for many helpful suggestions and improvements.

AI assistance. Portions of this document were prepared with the assistance of two large-language-model systems: *Claude Sonnet 4.5, 4.6, and 4.7* (Anthropic) and *Scholar GPT 5.4 Thinking* (OpenAI). The tools were used for:

- literature and standards triage;
- locating, extracting, and cross-checking benchmark figures for ML-KEM, ML-DSA, FN-DSA/Falcon, and Classic McEliece from primary sources and benchmark logs;
- drafting, editing, and restructuring explanatory prose and comparative tables;
- converting between Markdown and L^AT_EX and generating figure/table boilerplate; and
- brainstorming wording for the security discussion and email-style summaries.

In this revision, AI assistance was also used to normalize the attached liboqs/Falcon benchmark log to the paper’s 3.0 GHz reference convention and to propagate the corrected FN-DSA rows through the derived comparison tables. AI also wrote this acknowledgment.

All technical claims, protocol interpretations, numerical assumptions, and final editorial judgments were made by the human authors, who remain solely responsible for the content. AI-generated text, citations, and computations were reviewed, checked against the cited sources, corrected where necessary, and substantially revised before inclusion.

References

- [1] C. Kaufman et al., “Internet Key Exchange Protocol Version 2 (IKEv2),” RFC 7296, 2014. <https://www.rfc-editor.org/rfc/rfc7296>
- [2] C. J. Tjhai et al., “Multiple Key Exchanges in IKEv2,” RFC 9370, 2023. <https://www.rfc-editor.org/rfc/rfc9370>
- [3] V. Smyslov, “Internet Key Exchange Protocol Version 2 (IKEv2) Message Fragmentation,” RFC 7383, November 2014. <https://www.rfc-editor.org/rfc/rfc7383>
- [4] K. Kampanakis et al., “Post-quantum Hybrid Key Exchange with ML-KEM in IKEv2,” Internet-Draft draft-ietf-ipsecme-ikev2-mlkem-03, 2025. <https://datatracker.ietf.org/doc/draft-ietf-ipsecme-ikev2-mlkem/>
- [5] G. Wang et al., “KEM-Based Authentication for IKEv2,” Internet-Draft draft-wang-ipsecme-kem-auth-ikev2-03, IETF IPSECME WG. <https://datatracker.ietf.org/doc/draft-wang-ipsecme-kem-auth-ikev2/>
- [6] S.-L. Gazdag, S. Grundner-Culemann, T. Guggemos, T. Heider, D. Loebenberger, “A Formal Analysis of IKEv2’s Post-Quantum Extension,” *ACSAC 2021*, December 2021. <https://dl.acm.org/doi/10.1145/3485832.3485885>
- [7] NIST, “ML-KEM (FIPS 203),” August 2024. <https://csrc.nist.gov/pubs/fips/203/final>
- [8] NIST, “ML-DSA (FIPS 204),” August 2024. <https://csrc.nist.gov/pubs/fips/204/final>
- [9] NIST, “FN-DSA (FIPS 206, draft),” 2025. <https://csrc.nist.gov/pubs/fips/206/ipd>
- [10] Open Quantum Safe, “liboqs,” GitHub repository. <https://github.com/open-quantum-safe/liboqs>
- [11] D. J. Bernstein et al., “Classic McEliece,” Round 4, October 2022. <https://classic.mceliece.org>
- [12] D. J. Bernstein et al., “libmceliece speed table,” 2025-05-06. <https://lib.mceliece.org/speed.html>
- [13] wolfSSL, “Benchmark Appendix G,” Intel x86-64, AVX2. <https://www.wolfssl.com/documentation/manuals/wolfssl/appendix07.html>
- [14] CRYSTALS team, “pq-crystals/dilithium,” SUPERCOP, KabyLake. <https://github.com/pq-crystals/dilithium>
- [15] P.-J. Chen et al., “Complete and Improved FPGA Implementation of Classic McEliece,” CHES 2022. <https://eprint.iacr.org/2022/412>
- [16] B. Halak et al., “Benchmarking NIST-Standardized ML-KEM and ML-DSA on ARM Cortex-M0+ (RP2040),” arXiv, 2025. <https://arxiv.org/abs/2603.19340>
- [17] “Design and Implementation of an Authenticated Post-Quantum Session Protocol,” *Frontiers in Physics*, November 2025. <https://doi.org/10.3389/fphy.2025.1723966>
- [18] H. Krawczyk, “HMQV: A High-Performance Secure Diffie-Hellman Protocol,” CRYPTO 2005. IACR ePrint 2005/176. <https://eprint.iacr.org/2005/176>

- [19] S. Blake-Wilson, D. Johnson, A. Menezes, “Key Agreement Protocols and Their Security Analysis,” *LNCS*, December 1997. <https://doi.org/10.1007/BFB0024447>
- [20] U. Blumenthal et al., “PQuAKE — Post-Quantum Authenticated Key Exchange,” June 2026. <https://datatracker.ietf.org/doc/html/draft-uri-cfrg-pquake-00>
- [21] D. Stebila, P. Schwabe, T. Wiggers, “Post-Quantum TLS without Handshake Signatures,” *ACM CCS 2020. IACR ePrint 2020/534*. <https://ia.cr/2020/534>
- [22] D. Stebila, P. Schwabe, T. Wiggers, “More Efficient KEMTLS with Pre-Shared Keys,” *ESORICS 2021. IACR ePrint 2021/779*. <https://ia.cr/2021/779>
- [23] S. Celi, J. Hoyland, D. Stebila, T. Wiggers, “A Tale of Two Models: Formal Verification of KEMTLS in Tamarin,” *ESORICS 2022. IACR ePrint 2022/1111*. <https://ia.cr/2022/1111>
- [24] T. Wiggers et al., “KEM-based Authentication for TLS 1.3 (AuthKEM),” draft-celi-wiggers-tls-authkem-06, November 2025. <https://datatracker.ietf.org/doc/draft-celi-wiggers-tls-authkem/>
- [25] T. Wiggers et al., “KEM-based PSK Handshakes for TLS 1.3 (AuthKEM-PSK),” draft-wiggers-tls-authkem-psk-04, November 2025. <https://datatracker.ietf.org/doc/draft-wiggers-tls-authkem-psk/>
- [26] T. Wiggers, “Post-Quantum TLS,” PhD thesis, January 2024. <https://thomwiggers.nl/publication/thesis/>
- [27] A. Koranga, “Benchmarking Different PQC Libraries,” LinkedIn post, 2025. https://www.linkedin.com/posts/aditya-koranga_pqc-cupqc-postquantumcryptography-activity-7363632983175548929-dkWs/
- [28] S. Fluhrer, P. Kampanakis, D. McGrew, V. Smyslov, “Mixing Preshared Keys in the Internet Key Exchange Protocol Version 2 (IKEv2) for Post-quantum Security,” *RFC 8784*, June 2020. <https://www.rfc-editor.org/rfc/rfc8784>
- [29] S. Fluhrer, D. McGrew, P. Kampanakis, V. Smyslov, “Mixing Preshared Keys in the Internet Key Exchange Protocol Version 2 (IKEv2) for Post-quantum Security,” *RFC 9867*, January 2025. <https://www.rfc-editor.org/rfc/rfc9867> (*Supersedes RFC 8784; allows PPK application at IKE_INTERMEDIATE stage to protect auth-KEM exchanges.*)
- [30] NIST, “Recommendation for Key Management: Part 4 — Post-Quantum Cryptography (Draft),” Special Publication 800-227 (Draft), 2025. <https://csrc.nist.gov/publications/detail/sp/800-227/draft>
- [31] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997. <https://doi.org/10.1137/S0097539795293172>
- [32] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, Z. Zhang, “Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU,” NIST PQC Round 3/4 Submission, 2020. <https://falcon-sign.info/falcon.pdf>
- [33] R. J. McEliece, “A Public-Key Cryptosystem Based On Algebraic Coding Theory,” DSN Progress Report 42–44, Jet Propulsion Laboratory, pp. 114–116, 1978. <https://ipnpr.>

jpl.nasa.gov/progress_report2/42-44/44N.PDF

- [34] Battarbee et al., “Quantum-safe hybrid key exchanges with KEM-based authentication,” EPJ Quantum Technology, 12:128, 2025. <https://link.springer.com/article/10.1140/epjqt/s40507-025-00425-3>

A. Benchmark Data Sources

This appendix documents the primary benchmark data sources used throughout this analysis. All performance figures are normalized to a reference platform of Intel x86-64 with AVX2 at 3.0 GHz.

A.1 ML-DSA and FN-DSA: Local liboqs Measurements

The ML-DSA (Dilithium2/3/5) and FN-DSA (Falcon-512/1024) benchmark figures used in this document are derived from local measurements of the Open Quantum Safe liboqs library [10]. The measurements were performed using the `speed_sig` benchmark utility on the following platform:

- **Platform:** x86_64-Darwin-22.5.0 (macOS)
- **Library:** liboqs version 0.8.1-dev
- **Compiler:** clang version 16
- **Optimizations:** AVX2 enabled
- **CPU:** Intel processor with AVX2 support

The raw benchmark output is reproduced below. Cycle counts have been normalized to 3.0 GHz for consistency with other data sources in this document. The conversion formula is:

$$\text{Latency } (\mu\text{s}) = \frac{\text{Cycles}}{3,000,000} \times 10^6 = \frac{\text{Cycles}}{3,000}$$

A.2 Raw Benchmark Output

The following is the verbatim output from the liboqs `speed_sig` benchmark. Note that the original output reports cycles at the native CPU frequency; all figures in the main document have been scaled to 3.0 GHz.

Speed test
=====

Configuration: x86_64-Darwin-22.5.0
Compiler: clang version 16.0.0
OQS version: liboqs 0.8.1-dev

Algorithm	keygen	sign	verify
	avg (cy)	avg (cy)	avg (cy)
Dilithium2	65,563	167,361	61,392
Dilithium3	112,440	272,595	101,799
Dilithium5	190,290	362,451	164,055
Falcon-512	16,627,157	635,726	118,375
Falcon-1024	49,088,028	1,308,057	235,577

Notes:

- Dilithium2/3/5 correspond to ML-DSA-44/65/87 in FIPS 204
- Falcon-512/1024 correspond to FN-DSA-512/1024 in FIPS 206 (draft)
- Cycle counts are for the specified platform and may vary on other microarchitectures
- KeyGen operations for both algorithms are performed offline and do not contribute to per-session handshake cost

A.3 Interpretation and Usage

ML-DSA (Dilithium). The measured cycle counts are higher than optimized AVX2 implementations from the pq-crystals repository [14], which use SUPERCOP-style benchmarking on KabyLake processors. This document presents **both** the optimized estimates (used as the baseline in performance comparisons) and the measured liboqs data (shown in Table P-4, Section 5) to provide readers with a realistic performance range.

FN-DSA (Falcon). Falcon’s performance is highly implementation-dependent due to:

- Choice of Gaussian sampling algorithm (cumulative distribution table vs. rejection sampling)
- Floating-point vs. integer-only arithmetic
- Platform-specific FPU performance

The measured KeyGen times (16.6 M cycles for Falcon-512, 49.1 M cycles for Falcon-1024) reflect the cost of NTRU lattice basis generation, which is performed offline. The per-session signing cost (635k–1.3M cycles) is the relevant metric for handshake performance comparison.

Normalization to 3.0 GHz. All cycle counts in the main document tables are normalized to 3.0 GHz using direct cycle count scaling. This is valid for modern out-of-order superscalar processors where cycle counts are clock-speed-independent for compute-bound operations. For example:

- Dilithium3 (ML-DSA-65) Sign: $272,595 \text{ cycles} \div 3,000 = 90.9 \mu\text{s}$
- Falcon-512 (FN-DSA-512) Sign: $635,726 \text{ cycles} \div 3,000 = 212 \mu\text{s}$

A.4 Other Data Sources

In addition to the liboqs measurements above, this document uses:

- **ML-KEM:** wolfSSL Manual Appendix G [13] (Intel x86-64 AVX2, 1,000-run average)
- **ML-DSA (optimized):** pq-crystals/dilithium repository [14] (SUPERCOP cycle counts, KabyLake ~3.6 GHz, scaled to 3.0 GHz)
- **Classic McEliece:** libmceliece speed table [12] (Golden Cove, 2021; interquartile means of single-core cycle counts)

Note: we did not compare GPU, FPGA, or other special hardware accelerators, such as those described in [15] and [27]; those platforms lie outside the scope of this handshake-latency analysis. [27] provides newer and more diverse benchmarking results across a wider set of PQC libraries and may serve as a useful complement to the figures presented here.

All sources are cited in the main bibliography.